Planning at the Speed of Control

Profiling and fusing world-model robot planning into a single GPU kernel

Antoine Nguyen · **24433599**
Project presentation · [Faculty / School], University of Technology Sydney
Supervisor: [supervisor name] · October 2026

The project in one slide

- LeWM is a small AI world model that lets a robot plan by imagining thousands of possible futures. One plan still takes 622 ms on an H100 GPU.
- Question: where does that time go, and how much can be removed without retraining the model or making the robot less successful?
- Method: an analytics pipeline over 11,942 timed calls and 900 episode outcomes: profile, model the bottleneck, design, evaluate with paired statistics.
- Answer: the time was overhead, not computation. One fused GPU program per plan takes 46.5 ms, with task success unchanged.

622 ms
reference planner
(one robot, H100)

46.5 ms
LeWM-MK,
one GPU launch

13.4×
faster than the
reference

–0.3 pts
success vs reference,
300 paired episodes

Medians of end-to-end planning latency; 95% CI of the speed-up 13.0–13.5×

Why it matters: robots plan against a clock

- A world model predicts what happens next; a robot plans by imagining many action sequences and executing the best.
- A controller must finish each plan within its control period: 100 ms at 10 Hz, 50 ms at 20 Hz, for sensing, planning and acting together.
- LeWM made world-model planning cheap by shrinking the model to 15M parameters. Its released planner still takes 622 ms per plan.
- The model is tiny: its arithmetic per plan should take milliseconds. Something else is using the time.

15M
parameters in the whole model

9,000
imagined futures per reference plan

900
dependent layer evaluations per plan

16–53 s
per plan for larger world models

Aim and objectives

01 Profile: build a stage-by-stage compute profile of the reference planner from measured data, and identify the bottleneck.

02 Design and implement: remove the bottlenecks without retraining the model or shrinking the planner's search budget.

03 Evaluate: latency and task success against the reference and a strong optimised baseline, on the same GPU, with paired tests.

04 Deploy: a planner that plugs into the existing harness, and one program that regenerates every result from the data.

Data analytics and AI. The object is an AI system (a self-supervised world model and its search algorithm); the method is an analytics pipeline over its measured behaviour: collection, preparation, pattern discovery with a fitted latency model, statistical evaluation and deployment.

Background: three lines of work, one gap

World models that plan

JEPA models (I-JEPA, DINO-WM, V-JEPA 2, LeWM) predict in a learned representation and plan by search. Planning time is reported coarsely and only ever cut by changing the model.

Sampling-based search

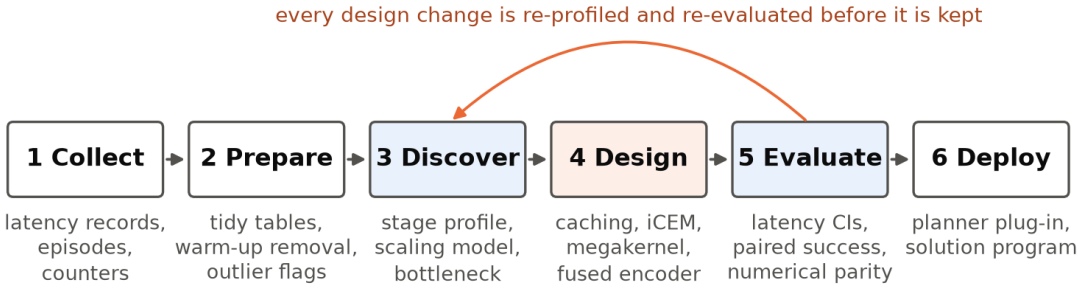
CEM samples, keeps the best, refits, repeats. iCEM shrinks the population and reuses the best: 2.7–22× fewer samples. But fewer samples need not mean less time (CEM-GD).

Fast GPU inference

Small models at batch 1 are bound by launch overhead and data movement. Remedies: CUDA Graphs, compilers, FlashAttention, and megakernels: one GPU program for a whole computation.

The gap. No one has profiled where a world-model planner's time goes, measured whether sample-efficient search saves time in practice, or fused a whole planning loop, with a neural encoder, rollouts, scoring and elite selection, into one GPU program.

Method: an analytics pipeline over the planner



Data

11,942 timed policy calls (61 one-robot episodes × 3 systems + 4 probes), 900 episode outcomes, telemetry, component timings.

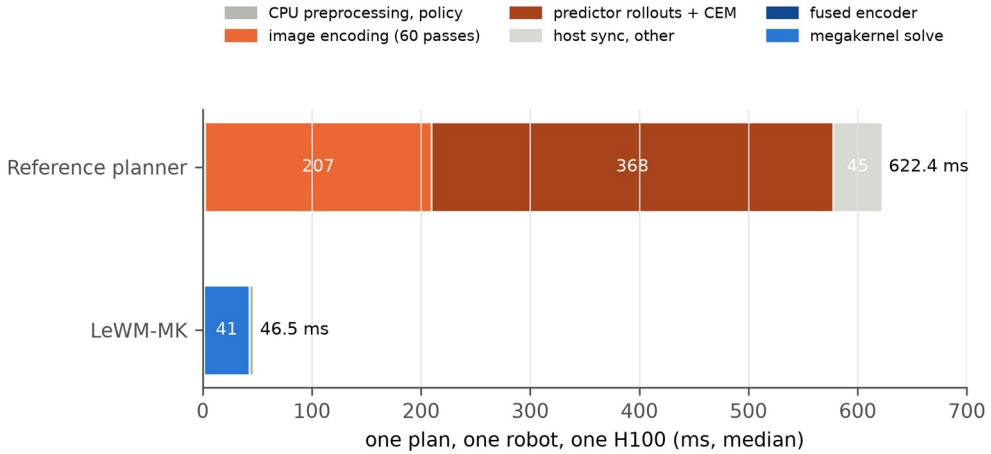
Analysis

Warm-up removal and outlier flags; stage decomposition; a fitted model latency = a + b·N separating fixed overhead from work.

Evaluation

Medians with bootstrap CIs; 300 paired episodes, McNemar and paired bootstrap; numerical error against FP32.

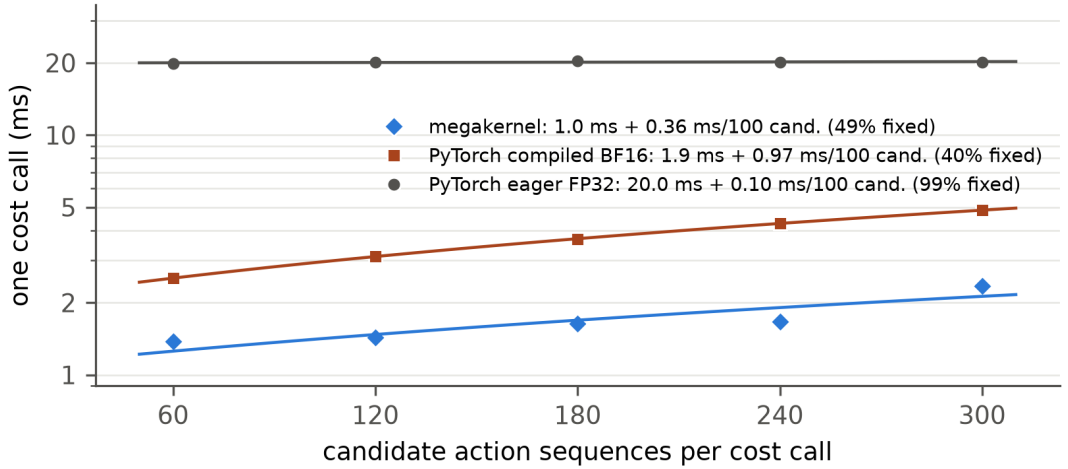
Finding: where a plan's 622 ms goes



Top bar: reference planner, by stage. Bottom bar: LeWM-MK (Section 6). Stages without a direct measurement attributed by difference.

- 33%: re-encoding the same two images 60 times per plan (telemetry: 60 encoder calls, 2 distinct images).
- 59%: the search, 12.3 ms per iteration, 409 µs per layer for about 3 µs of arithmetic.
- 7%: waiting on the CPU between iterations.
- Under 1%: the policy code around the planner.

Pattern: the search is overhead, not work



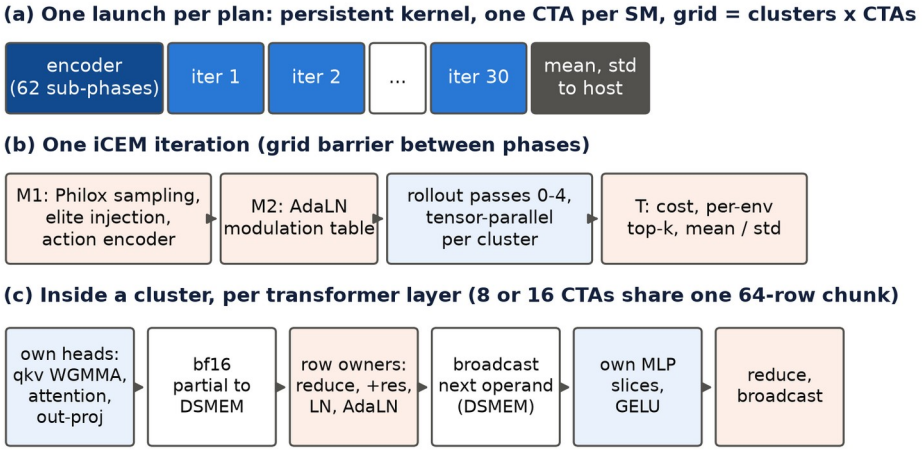
Latency of one scoring call vs number of candidates, with fitted lines a + b·N.

99%
of an eager call is fixed cost

1.01×
predicted iCEM gain, eager

- Fitted model: 20.0 ms fixed + 0.10 ms per 100 candidates. Cost does not depend on how much work is done.
- So cutting candidates (iCEM) cannot help here: predicted 1.01×, observed 0.92×.
- Only an execution with little fixed cost turns fewer candidates into less time.

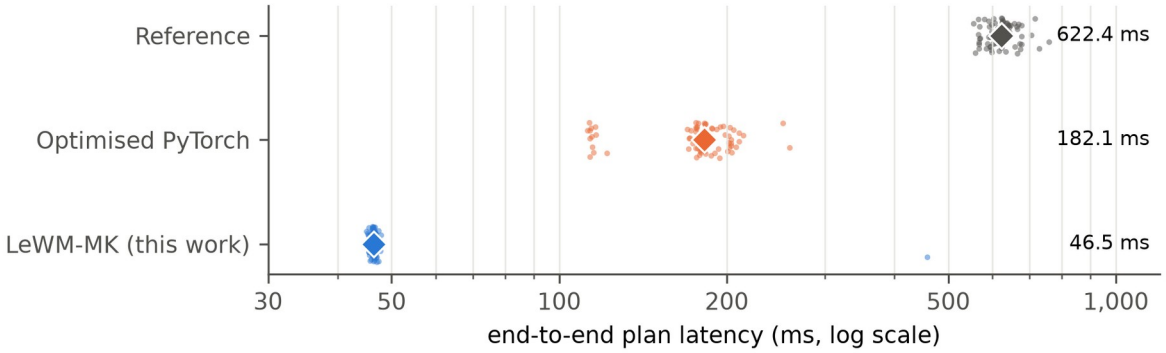
Solution: LeWM-MK, one GPU program per plan



Weights: 1,472 pre-swizzled 24 KB bf16 panels (with encoder). Each warpgroup's thread 0 refills its 2-slot shared-memory ring with cp.async.bulk from a static schedule, so weight prefetch runs through op, layer, pass and barrier boundaries.

- Encode each image once; cache the goal for the episode.
- iCEM search: 2,820 instead of 9,000 imagined futures.
- Whole plan in one persistent kernel: sampling, rollouts, scoring, elite selection, encoder; the CPU waits once.
- Small model spread across a 16-processor cluster; partial results exchanged on chip; barrier 3.6 → 0.65 µs.
- Weights streamed ahead of use; BF16 inputs, FP32 accumulation.

Evaluation: 622 ms to 46.5 ms on the same GPU



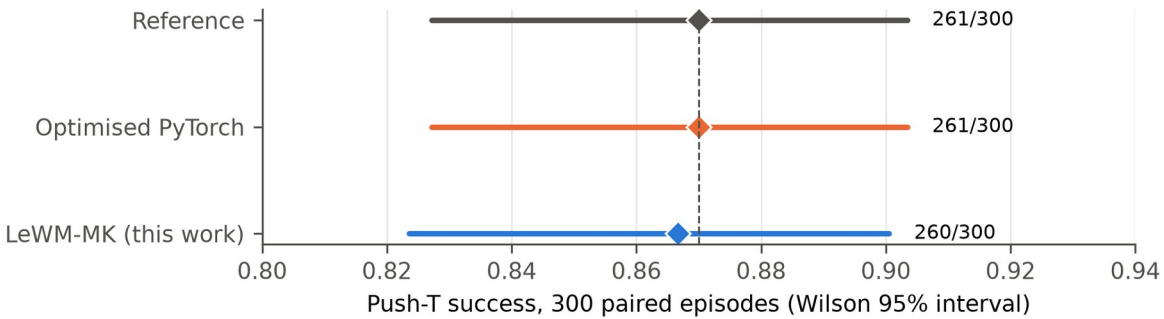
End-to-end planning latency, log scale; points are plans, diamonds medians. Baseline already uses compile, CUDA Graphs and BF16.

13.4×
vs reference
[13.0, 13.5]

3.9×
vs optimised
PyTorch

- 90th percentile 47.0 ms; interquartile range 0.7 ms wide.
- Fits a 20 Hz control loop at the median.
- Search now 46 µs per layer (was 409); synchronisation about 7% of it.

Evaluation: the robot succeeds just as often



Push-T success on the same 300 episodes, Wilson 95% intervals. 300 pairs detect differences of about 6 points.

- 261 / 261 / 260 successes of 300 for reference, optimised PyTorch and LeWM-MK.
- LeWM-MK vs reference: 19 won, 20 lost; −0.3 points [−4.3, +3.7]; McNemar p = 1.00.
- Disagreements are search randomness: the optimised PyTorch planner disagrees on 36.
- Numerical error vs FP32: 0.06–0.32%, matching accepted BF16 (0.05–0.40%).

Conclusions, and what I cannot claim

01 The reference's time was overhead: 33% redundant encoding, a search 99% fixed cost per call, 7% CPU waits.

02 LeWM-MK removes each bottleneck with the model and search budget unchanged.

03 46.5 ms per plan: 13.4× the reference, 3.9× optimised PyTorch; success unchanged (−0.3 pts).

04 Planner plug-in; one program regenerates every result from the data.

Limits

- 300 pairs resolve about 6 points of success, no less.
- Some profile stages attributed by difference.
- One model, one task, one GPU type; H100-only kernel.
- Simulation; only the planning call optimised.
- PyTorch timings vary up to 18% between passes; the kernel's about 2%.

Future work

1. **Spend the saved time on better plans**
   More iterations or candidates still fit a 100 ms period: map the latency-success trade-off.

2. **Shorten the remaining chain**
   Combine with a one-pass rollout model, or one layer per cluster with candidates streaming through.

3. **Other models and hardware**
   Larger world models, onboard robot computers; automatic megakernel compilers.

4. **Evaluate at deployment resolution**
   About 2,500 paired episodes for 2 points, now minutes of planning; then a physical robot.

Take-home

- Profile before optimising: the world-model planner's time was overhead, not arithmetic.
- A one-line fitted model of latency against workload told which fix would pay, and why the smarter search had looked useless.
- One GPU program per plan: 622 ms to 46.5 ms on the same H100, same model, same task success.

Report, datasets and solution program: docs/paper and solution_pack/ in the project repository.